Axlero Data Analytics (DA) Internship: Exhaustive Project Requirements and Analytical Specifications

The transition from academic data analysis to enterprise-level data science demands a rigorous paradigm shift. Modern product-based organizations require analysts who not only write functional Pandas syntax but who can architect scalable data pipelines, translate abstract business requirements into robust statistical models, and maintain strict version control discipline. As part of the Axlero digital transformation and startup acceleration initiatives, this document serves as the master Project Requirements Document (PRD) repository for the Data Analytics (DA) engineering internship.

This technical specification details three exhaustive, high-impact startup projects, each mapped to a strict four-week development lifecycle. The systems are designed to address real-world business needs in Retail/E-commerce, Finance, and Supply Chain. The engineering expectations simulate the environment of top-tier technology firms, emphasizing data integrity, reproducible research, and rigorous peer review.

Universal Architectural Directives and Engineering Standards

Before addressing the specific requirements of the data projects, it is imperative to establish the foundational analytical standards that govern all deliverables. Adherence to these protocols is non-negotiable and forms the baseline for the internship evaluation.

The Technology Stack (Data Analytics)

All projects must be engineered using a modern, scalable data stack. The mandated stack is:

- **Programming Language:** Python (3.9+)
- **Data Manipulation & Analysis:** Pandas, NumPy, SciPy
- **Database & Querying:** PostgreSQL (utilizing SQLAlchemy or Psycopg2 for connections)
- **Machine Learning:** Scikit-Learn, Statsmodels
- **Visualization & Reporting:** Matplotlib, Seaborn, and an external BI tool (Tableau or PowerBI)
- **Environment Management:** Conda or standard Python `venv` with a strict `requirements.txt`

Standardized Project Structure (Cookiecutter Data Science)

The most critical architectural mandate for the data analytics internship is the strict implementation of reproducible research principles. Ad hoc scripts and messy Jupyter notebooks scattered across a desktop are unacceptable. All projects must adhere to the industry-standard Cookiecutter Data Science directory structure, which ensures analysis is reproducible, maintainable, and logically separated.

The mandated directory structure for all data repositories is outlined below:

Directory Path	Purpose and Implementation Guidelines
data/	Organizes datasets. Must contain subfolders: raw/ (original, immutable data dump), interim/ (transformed data), and processed/ (final canonical datasets for modeling).
notebooks/	Houses Jupyter notebooks. Must follow a numbered naming convention for ordering (e.g., 1.0-initial-exploration, 2.0-feature-engineering).
src/	Reusable Python source code. Includes dataset.py (data generation), features.py (feature engineering), and modeling/ (training and prediction scripts).
reports/	Contains generated analysis as HTML, PDF, or Markdown, along with a figures/ subfolder for generated graphics used in reporting.
requirements.txt	The requirements file for reproducing the analysis environment exactly as it was built.

Version Control Discipline and Evaluation Criteria

A core component of professional analytics is the ability to communicate changes clearly through version control. Interns are expected to operate exactly as they would in a distributed, enterprise data team. This requires abandoning the academic habit of pushing a massive block of code or a single bloated Jupyter notebook at the end of a project. Instead, interns must utilize continuous, atomic commits that provide a clear historical record of the analytical evolution.

All repositories must strictly implement clear commit messages detailing what data manipulation, model tweak, or visualization was added.

Mandatory Evaluation Criterion: The evaluation will only be done if the intern is having all 4 weeks of git hub commits and contributions. A repository demonstrating a lack of daily, iterative commits, or a massive single push at the conclusion of the timeline, will result in immediate disqualification from the program.

Project 1: Retail & Ecommerce – Customer Segmentation and Churn Prediction

System Context and Problem Statement

Axlero's D2C (Direct-to-Consumer) retail partners frequently suffer from high customer acquisition costs combined with unpredictable retention rates. Without understanding which customer segments are highly valuable and which are at risk of leaving, marketing budgets are wasted. This project requires the DA intern to analyze transactional data, segment the customer base using RFM (Recency, Frequency, Monetary) metrics, and build a predictive model to identify customers highly likely to churn in the next 30 days.

System Objectives and Success Metrics

Metric Category	Target Objective
Data Integrity	Missing values and outliers in the transactional data must be handled programmatically without dropping more than 5% of the total dataset.
Model Performance	The Churn Prediction model must achieve a minimum ROC-AUC score of 0.75.
Actionability	The final BI dashboard must clearly define at least 4 distinct customer segments with actionable marketing recommendations for each.

Four-Week Development Lifecycle

Week 1: Data Ingestion & Exploratory Data Analysis (EDA)

- **Engineering Focus:** SQL extraction, Pandas cleaning, and initial distributions.
- **Technical Implementation Details:** Initialize the Cookiecutter Data Science folder structure. Load raw transactional CSVs into a local PostgreSQL instance. Write SQL queries to extract flattened user-level data. Use Jupyter Notebooks (1.0-eda) to analyze distributions, handle missing values, and visualize sales trends.
- **Deliverables:** Cleaned data/interim datasets, documented EDA notebook, and daily GitHub commits.

Week 2: RFM Feature Engineering & Unsupervised Learning

- **Engineering Focus:** Metric calculation and K-Means clustering.

- **Technical Implementation Details:** Develop src/features.py to calculate Recency, Frequency, and Monetary values for every unique customer. Standardize the data using Scikit-Learn. Apply K-Means clustering to create distinct customer segments. Determine the optimal number of clusters using the Elbow Method or Silhouette Score.
- **Deliverables:** Segmented data/processed dataset, clustering visualization notebook, and continuous Git commits.

Week 3: Supervised Modeling (Churn Prediction)

- **Engineering Focus:** Classification algorithms and evaluation metrics.
- **Technical Implementation Details:** Define the "Churn" target variable (e.g., no purchase in the last 60 days). Split data into train/test sets. Train Logistic Regression and Random Forest classifiers. Evaluate using Precision, Recall, F1-Score, and ROC-AUC. Address class imbalance using SMOTE or class weights if necessary.
- **Deliverables:** Trained model files (Pickle/Joblib) saved in the models/ directory, evaluation metrics report, and continuous daily commits.

Week 4: Business Intelligence Dashboard & Final Reporting

- **Engineering Focus:** Stakeholder communication and visual storytelling.
- **Technical Implementation Details:** Connect Tableau or PowerBI to the data/processed outputs. Build an interactive dashboard displaying overall revenue, segment breakdowns, and a "High-Risk Churn" list. Write a comprehensive README.md and final Markdown report in reports/ summarizing business recommendations.
- **Deliverables:** Completed repository, exported BI dashboard file (.twbx or .pbix), and a pristine, 4-week Git history.

Project 2: Finance & Banking – Credit Risk Scoring & Loan Default Analysis

System Context and Problem Statement

For Axlero's fintech clients, assessing the creditworthiness of applicants is critical for minimizing financial risk. Traditional rule-based lending often rejects good candidates or approves risky ones. This project requires the intern to perform deep statistical analysis on historical loan data to identify the key drivers of default and build a robust Credit Risk Scoring model.

System Objectives and Success Metrics

Metric Category	Target Objective
Statistical Rigor	Features must be selected using statistical tests (e.g., Chi-Square for categorical, ANOVA for continuous variables).

Model Interpretability	The model cannot be a "black box"; feature importance must be clearly calculated and presented to simulate regulatory compliance.
Pipeline Automation	Data transformation steps must be wrapped in a Scikit-Learn `Pipeline` object for seamless inference.

Four-Week Development Lifecycle

Week 1: Data Auditing & Complex Imputation

- **Engineering Focus:** Assessing data quality and handling financial outliers.
- **Technical Implementation Details:** Set up the project structure. Ingest historical loan data. Perform univariate and bivariate analysis. Handle missing financial variables (like income or debt-to-income ratio) using advanced techniques like KNN-Imputation rather than simple mean imputation.
- **Deliverables:** Documented data audit in `notebooks/`, clean `data/interim` files, and atomic Git commits.

Week 2: Advanced Feature Engineering (WoE & IV)

- **Engineering Focus:** Financial risk metrics and dimensionality reduction.
- **Technical Implementation Details:** Calculate Weight of Evidence (WoE) and Information Value (IV) for categorical variables to determine their predictive power for loan defaults. Group continuous variables into logical bins to stabilize the model. Add these transformation rules to `src/features.py`.
- **Deliverables:** Feature importance ranking notebook, finalized `data/processed` set, and continuous Git commits.

Week 3: Predictive Modeling & Threshold Tuning

- **Engineering Focus:** Gradient Boosting, hyperparameter tuning, and business logic thresholds.
- **Technical Implementation Details:** Train an XGBoost or LightGBM model. Utilize GridSearchCV or RandomizedSearchCV to optimize hyperparameters. Critically, analyze the Precision-Recall curve to select an optimal classification threshold that minimizes false negatives (approving a bad loan) based on simulated business costs.
- **Deliverables:** Tuned model, threshold analysis charts saved to `reports/figures/`, and continuous daily commits.

Week 4: Model Explainability (SHAP) & Executive Presentation

- **Engineering Focus:** SHAP values and stakeholder reporting.
- **Technical Implementation Details:** Implement SHAP (SHapley Additive exPlanations) to interpret the XGBoost model outputs, generating summary plots and individual force plots to explain *why* a specific loan was rejected. Compile these visual findings into a final PDF report tailored for a non-technical risk committee.

- **Deliverables:** Completed Python scripts, SHAP visualization notebook, final executive report, and a meticulously maintained Git commit history.

Project 3: Logistics & Supply Chain – Delivery Route Delay Forecasting

System Context and Problem Statement

Axlero's logistics partners suffer financial penalties when SLA (Service Level Agreement) delivery times are breached. Accurately predicting which shipments are likely to be delayed before they leave the warehouse allows operational managers to intervene. This project requires analyzing spatio-temporal supply chain data to forecast delivery delays based on weather, route distance, vehicle type, and historical vendor performance.

System Objectives and Success Metrics

Metric Category	Target Objective
Temporal Handling	Time-series data must be properly formatted, accounting for seasonality, day-of-week, and time-of-day features.
Geospatial Processing	Distance metrics (e.g., Haversine distance) must be accurately calculated from raw latitude/longitude coordinates.
Forecasting Accuracy	The regression model predicting total delivery hours must achieve an R-squared of at least 0.65 or an explicitly minimized RMSE.

Four-Week Development Lifecycle

Week 1: Spatio-Temporal Data Wrangling

- **Engineering Focus:** Datetime manipulation and geospatial math.
- **Technical Implementation Details:** Initialize the required directory structure. Parse complex datetime strings into pandas `datetime` objects. Calculate delivery duration in hours. Use the math module to calculate Haversine distances between origin and destination coordinates.
- **Deliverables:** Base dataset with corrected timestamps and distances, initial distribution plots, and atomic Git commits.

Week 2: Time-Series EDA & External Variable Merging

- **Engineering Focus:** Identifying trends and enriching datasets.
- **Technical Implementation Details:** Analyze average delays by day of the week, month, and specific carrier. Create rolling averages for vendor performance. If applicable, simulate merging external weather data (e.g., precipitation flags) via Pandas `merge` or `join` operations in `src/dataset.py`.
- **Deliverables:** Enriched `data/processed` dataset, time-series visualization notebook, and continuous Git commits.

Week 3: Regression Modeling for Delay Prediction

- **Engineering Focus:** Supervised regression and error analysis.
- **Technical Implementation Details:** Transform categorical variables (Carrier ID, Vehicle Type) using One-Hot Encoding. Train a Random Forest Regressor and a Ridge Regression model to predict the continuous `hours_delayed` variable. Analyze the residual plots to ensure the model isn't systematically biased.
- **Deliverables:** Regression evaluation report (RMSE, MAE, R-squared), residual plot graphics, and continuous daily commits.

Week 4: Operational Dashboarding

- **Engineering Focus:** Translating predictions into operational tools.
- **Technical Implementation Details:** Load the predictions alongside the test set into Tableau/PowerBI. Create an "Operations Dashboard" that highlights routes with the highest predicted delay risk, allowing a dispatcher to filter by region or carrier. Finalize the `requirements.txt` file.
- **Deliverables:** Completed Python repository, fully functioning BI dashboard, and a pristine 4-week Git history.

Internship Evaluation Criteria and Final Deliverables

The ultimate goal of this internship framework is to evaluate analytical maturity, statistical capability, and workflow consistency. Axlero utilizes evaluation protocols mirroring those of top-tier technology firms, requiring interns to demonstrate both deep technical execution and the ability to articulate analytical decisions to stakeholders.

Deliverable Submission Guidelines

At the conclusion of the four-week sprint, interns must submit the following artifacts:

1. **Source Code Repository:** Hosted on GitHub, containing the complete Data Analytics project structured strictly via the Cookiecutter Data Science layout.
2. **BI Dashboard:** An exported Tableau (.twbx) or PowerBI (.pbix) file, or a hosted link if utilizing a cloud BI platform.
3. **Analytical Documentation (README.md & Reports):** A detailed overview of the system,

including setup instructions to recreate the Conda/Venv environment, and a finalized report in the `reports/` folder detailing the business insights.

The Hiring Manager (HM) Technical Presentation

Following the submission, candidates will undergo a rigorous technical presentation and defense. During this session, interns will be evaluated on:

- **Business Context:** The ability to describe the business problem concisely and how their analysis directly solves it.
- **Statistical Nuance:** Articulating why certain features were engineered, how data leakage was prevented during cross-validation, and the trade-offs of the chosen evaluation metrics (e.g., why Recall might be prioritized over Precision for churn).
- **Code Organization:** Defending the separation of concerns within their repository and demonstrating clean, PEP-8 compliant Python code.

The Non-Negotiable Git Contribution Mandate

Regardless of the quality of the final analytical submission, workflow discipline is paramount. The modern data science environment relies entirely on version control and reproducible workflows.

Final Warning: The evaluation will only be done if the intern is having all 4 weeks of individual 20 days github commits and contributions.

This signifies that the evaluation committee will heavily scrutinize the commit history graph. The commit history must reflect a steady, iterative progression of development—starting from data ingestion, moving through EDA, into modeling, and concluding with visualization. An intern who attempts to circumvent the four-week process by pushing the majority of the Jupyter notebooks and datasets in the final days will be deemed to lack the workflow maturity required for a Axlero data role, and the project will be rejected without a technical review.